

KIẾN TRÚC HỆ THỐNG PLAYNEST WEB GAME PLATFORM

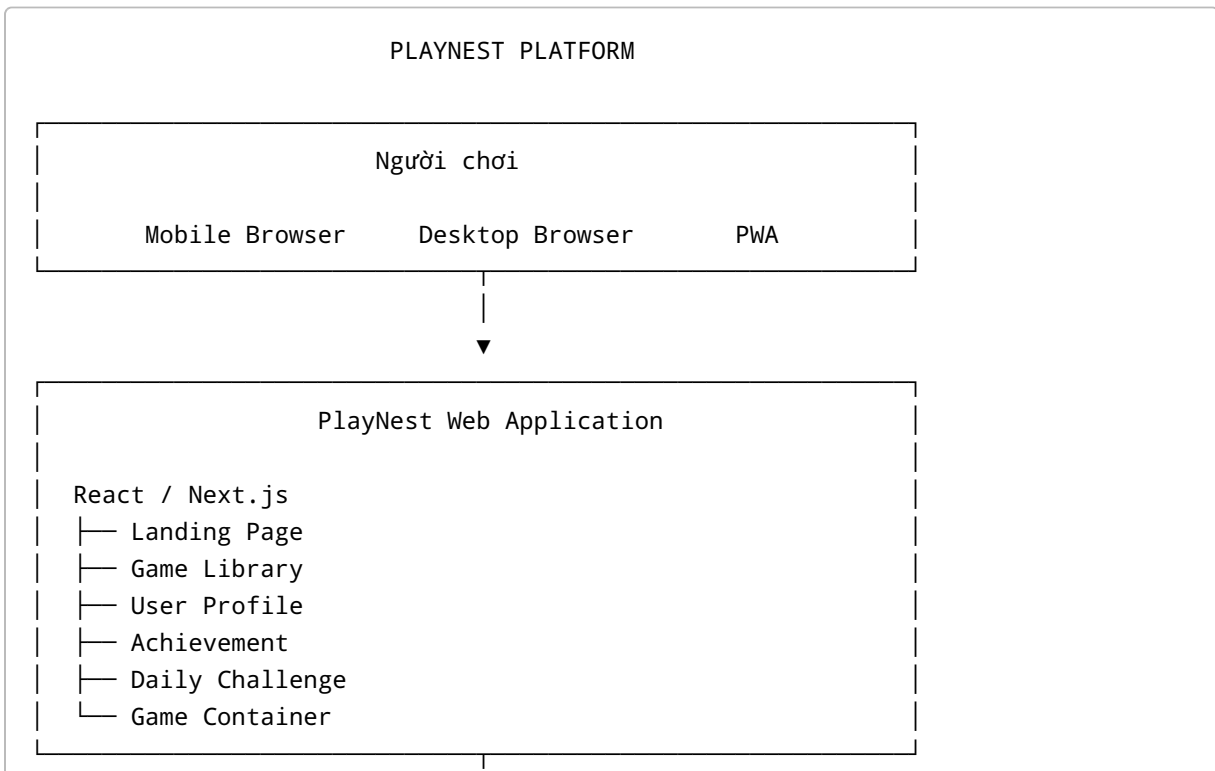
1. Mục tiêu hệ thống

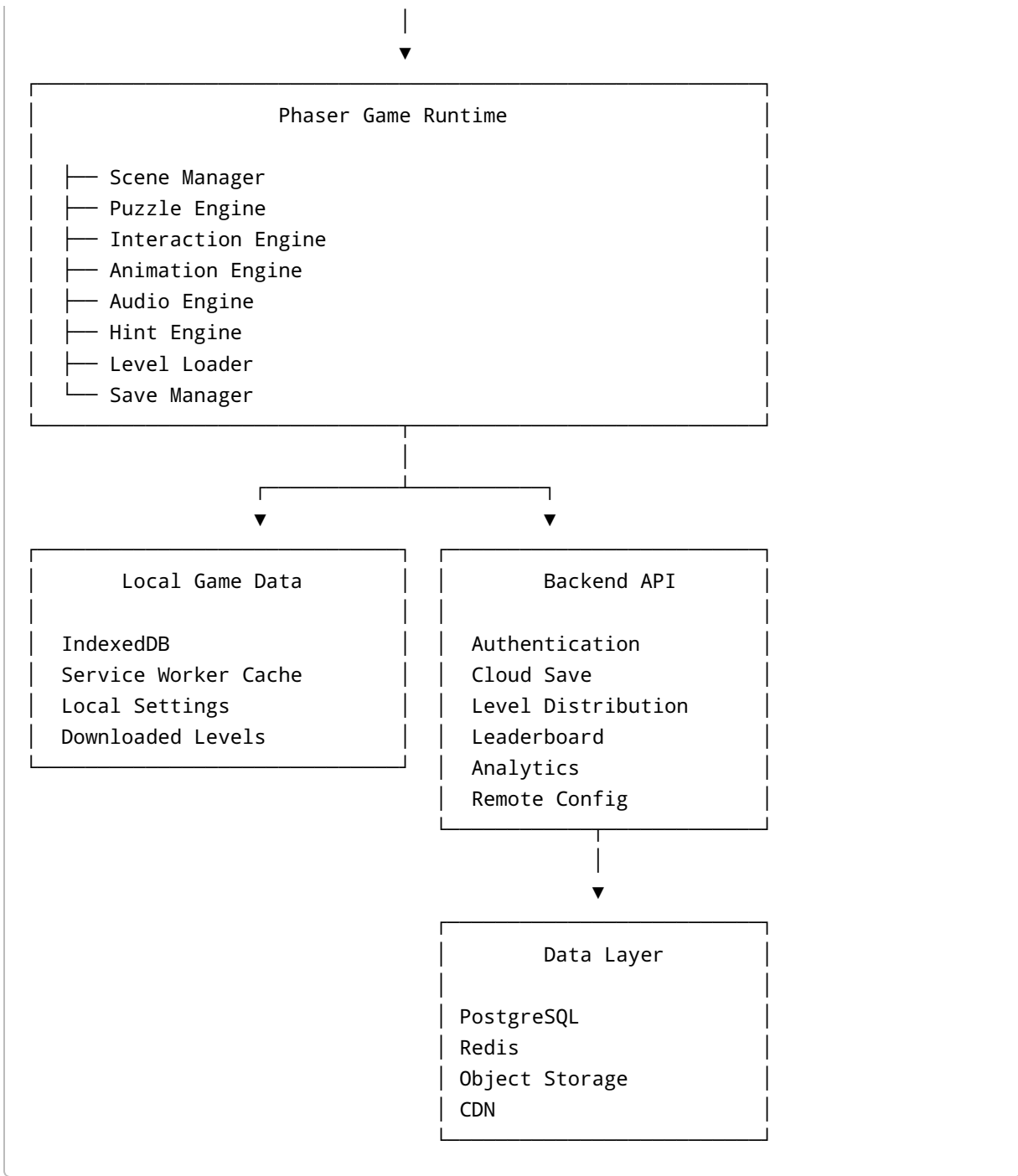
PlayNest là nền tảng web game casual, cho phép người dùng mở trình duyệt và chơi ngay mà không cần cài ứng dụng.

Giai đoạn đầu tập trung vào game giải đố kiểu Brain Test:

- Mỗi màn chơi kéo dài khoảng 5-30 giây.
- Hỗ trợ chạm, kéo, thả, giữ, xoay, phóng to và nhiều thao tác cùng lúc.
- Có thể chơi trên điện thoại, máy tính bảng và desktop.
- Có thể chơi offline sau lần tải đầu.
- Nội dung màn chơi được quản lý bằng dữ liệu.
- Có công cụ tạo màn chơi riêng cho designer.
- Có thể mở rộng thành hàng chục game dùng chung hạ tầng.
- Kiếm tiền bằng quảng cáo, gói bỏ quảng cáo và vật phẩm hỗ trợ.

2. Kiến trúc tổng quan





3. Công nghệ đề xuất

3.1 Frontend nền tảng

Next.js hoặc React
TypeScript
Tailwind CSS
TanStack Query
Zustand
PWA

Next.js phụ trách:

- Trang chủ PlayNest.
- Danh sách game.
- Trang chi tiết từng game.
- Tài khoản người dùng.
- Hồ sơ và thành tích.
- SEO.
- Trang chính sách.
- Khung chứa Phaser game.

Phaser chỉ phụ trách phần gameplay.

Không nên dùng React để điều khiển từng sprite trong game vì React không phù hợp với game loop liên tục.

3.2 Game Engine

Phaser 3
TypeScript
WebGL
Canvas fallback

Phaser phụ trách:

- Render sprite.
- Nhận sự kiện chuột và cảm ứng.
- Drag and drop.
- Animation.
- Tween.
- Particle nhẹ.

- Camera.
- Audio.
- Scene lifecycle.
- Scale đa màn hình.

Phaser phù hợp với Brain Test vì game chủ yếu là đồ họa 2D và tương tác đơn giản.

3.3 Backend

Có thể chọn một trong hai hướng:

Phương án TypeScript

```
NestJS
PostgreSQL
Redis
S3-compatible Storage
```

Ưu điểm:

- Đồng nhất TypeScript toàn hệ thống.
- Phát triển nhanh.
- Dễ tuyển lập trình viên.
- Dễ chia sẻ model và validation giữa frontend và backend.

Phương án Go

```
Go
Fiber hoặc Gin
PostgreSQL
Redis
S3-compatible Storage
```

Ưu điểm:

- Nhẹ.
- Hiệu năng tốt.
- Chi phí vận hành thấp.
- Phù hợp khi lượng người chơi lớn.

Giai đoạn đầu nên dùng NestJS để ra sản phẩm nhanh. Khi có lượng truy cập lớn mới cần tối ưu hoặc tách một số service sang Go.

4. Cấu trúc monorepo

```
playnest/  
├── apps/  
│   ├── web/  
│   │   ├── landing  
│   │   ├── game-library  
│   │   ├── profile  
│   │   └── game-container  
│   ├── api/  
│   │   ├── auth  
│   │   ├── users  
│   │   ├── games  
│   │   ├── levels  
│   │   ├── progress  
│   │   ├── leaderboard  
│   │   └── analytics  
│   ├── level-editor/  
│   │   ├── scene-editor  
│   │   ├── object-properties  
│   │   ├── interaction-builder  
│   │   ├── solution-builder  
│   │   └── level-preview  
│   └── admin/  
│       ├── dashboard  
│       ├── users  
│       ├── games  
│       ├── levels  
│       ├── monetization  
│       └── remote-config  
├── packages/  
│   ├── game-engine/  
│   ├── puzzle-engine/  
│   ├── interaction-engine/  
│   ├── level-schema/  
│   ├── analytics-sdk/  
│   ├── ads-sdk/  
│   ├── auth-sdk/  
│   ├── ui/  
│   └── shared-types/  
└──
```

```

├── shared-utils/
├── games/
│   ├── tricky-brain/
│   ├── word-puzzle/
│   ├── hidden-object/
│   ├── emoji-quiz/
│   └── sudoku/
├── infrastructure/
│   ├── docker/
│   ├── nginx/
│   ├── terraform/
│   ├── monitoring/
│   └── ci-cd/
└── docs/
    ├── architecture
    ├── level-format
    ├── game-design
    └── deployment

```

Nên dùng monorepo với:

```

pnpm workspace
Turborepo

```

Điều này giúp các game cùng dùng chung:

- Puzzle engine.
- SDK quảng cáo.
- Analytics.
- Save game.
- UI.
- User account.
- Kiểu dữ liệu.

5. Kiến trúc bên trong một game

```

TrickyBrainGame
├── BootScene

```

```
├─ PreloadScene
├─ HomeScene
├─ LevelSelectScene
├─ GameplayScene
├─ ResultScene
└─ SettingsScene
```

BootScene

Phụ trách:

- Khởi tạo game.
- Kiểm tra thiết bị.
- Đọc cấu hình.
- Khởi tạo analytics.
- Đọc dữ liệu lưu local.

PreloadScene

Phụ trách:

- Tải asset bắt buộc.
- Tải font.
- Tải audio.
- Tải level đầu tiên.
- Hiển thị progress.

GameplayScene

Phụ trách:

- Đọc dữ liệu level.
- Tạo object.
- Gắn interaction.
- Theo dõi trạng thái.
- Kiểm tra đáp án.
- Hiển thị animation thành công hoặc thất bại.
- Gửi analytics.

6. Puzzle Engine

Puzzle Engine là phần cốt lõi quan trọng nhất.

Thay vì viết code riêng cho từng level, engine cung cấp một tập hợp hành động dùng lại.

6.1 Các thao tác cơ bản

- Tap
- Double Tap
- Long Press
- Drag
- Drop
- Swipe
- Pinch
- Rotate
- Shake
- Hold
- Multi Touch
- Text Input
- Object Combination
- Device Orientation

6.2 Các điều kiện kiểm tra

- Object moved to target
- Object tapped N times
- Objects overlap
- Object rotated to angle
- Object scale reaches value
- Multiple objects selected
- Sequence completed
- Timer completed
- Variable equals value
- All conditions completed

6.3 Các hành động phản hồi

- Move object
- Rotate object
- Scale object
- Hide object
- Show object
- Change sprite
- Play animation
- Play sound
- Update text

- Set variable
- Spawn object
- Destroy object
- Complete level
- Fail attempt

7. Cấu trúc dữ liệu level

Mỗi level nên được lưu dưới dạng JSON.

Ví dụ:

```
{
  "id": "tricky-brain-001",
  "gameId": "tricky-brain",
  "version": 1,
  "question": {
    "text": "Hãy giúp chú mèo lấy được con cá",
    "position": {
      "x": 0.5,
      "y": 0.12
    }
  },
  "background": {
    "type": "color",
    "value": "#FFF5D6"
  },
  "objects": [
    {
      "id": "cat",
      "type": "sprite",
      "asset": "cat_hungry",
      "position": {
        "x": 0.2,
        "y": 0.65
      },
      "interactive": false
    },
    {
      "id": "fish",
      "type": "sprite",
      "asset": "fish",
      "position": {
        "x": 0.8,
```

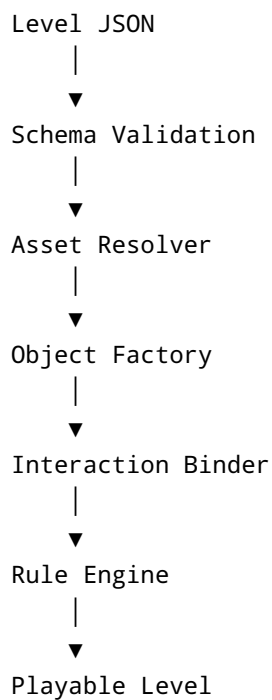
```

        "y": 0.65
      },
      "interactive": true,
      "interactions": [
        {
          "type": "drag"
        }
      ]
    }
  ],
  "variables": {
    "fishDelivered": false
  },
  "rules": [
    {
      "event": "drop",
      "source": "fish",
      "condition": {
        "type": "overlap",
        "target": "cat"
      },
      "actions": [
        {
          "type": "setVariable",
          "key": "fishDelivered",
          "value": true
        },
        {
          "type": "playAnimation",
          "target": "cat",
          "animation": "eat"
        },
        {
          "type": "completeLevel"
        }
      ]
    }
  ],
  "hint": {
    "text": "Hãy thử kéo con cá lại gần chú mèo."
  }
}

```

8. Level Loader

Level Loader nhận JSON và tạo level trong Phaser.



Thành phần chính

Schema Validator

Kiểm tra level có đúng cấu trúc hay không.

Có thể sử dụng:

Zod
JSON Schema
Ajv

Object Factory

Tạo object dựa trên loại:

Sprite
Text

Shape
Button
Container
Particle
Video

Interaction Binder

Gắn thao tác:

TapHandler
DragHandler
SwipeHandler
PinchHandler
RotateHandler
MultiTouchHandler

Rule Engine

Lắng nghe sự kiện và kiểm tra điều kiện.

9. Hệ thống tọa độ responsive

Không nên lưu tọa độ pixel tuyệt đối.

Nên lưu tọa độ chuẩn hóa từ 0 đến 1.

Ví dụ:

```
{  
  "x": 0.5,  
  "y": 0.7  
}
```

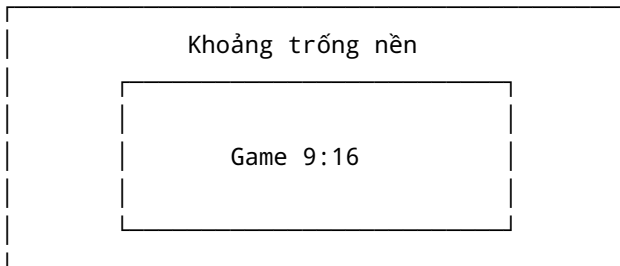
Điều này có nghĩa object nằm ở:

50% chiều rộng màn hình
70% chiều cao màn hình

Game sử dụng một canvas chuẩn:

Thiết kế gốc: 1080 × 1920
Tỉ lệ ưu tiên: 9:16

Trên desktop:



Game nên giữ tỉ lệ dọc giống mobile để designer chỉ cần tạo một layout.

10. Level Editor

Level Editor là yếu tố quyết định khả năng sản xuất hàng trăm hoặc hàng nghìn màn chơi.

10.1 Giao diện đề xuất

Toolbar		
Asset Panel	Game Canvas	Properties Panel
Cat		Position
Fish		Scale
Door		Rotation
Text		Interaction
Shape		Rules
		Animation
Timeline / Event / Rule Builder		

10.2 Chức năng editor

- Kéo asset vào canvas.

- Di chuyển, resize và rotate.
- Chỉnh layer.
- Đặt câu hỏi.
- Gắn hành động tương tác.
- Tạo vùng drop.
- Tạo điều kiện thẳng.
- Tạo animation.
- Viết hint.
- Preview ngay trong trình duyệt.
- Kiểm tra trên nhiều kích thước màn hình.
- Publish level.
- Quản lý phiên bản.

10.3 Rule Builder

Designer không cần viết code.

Ví dụ:

```

WHEN
    fish is dropped

IF
    fish overlaps cat

THEN
    play cat_eat animation
    play success sound
    complete level
  
```

Editor chuyển cấu hình này thành JSON.

11. Asset Management

Asset không nên nhúng toàn bộ vào source code.

Cấu trúc:

```

assets/

├─ common/
│   └─ ui/
│   └─ fonts/
  
```

```
| | | sounds/  
| | | particles/  
| |  
| | | tricky-brain/  
| | | | characters/  
| | | | objects/  
| | | | backgrounds/  
| | | | animations/  
| |  
| | | word-puzzle/
```

Asset được tải lên Object Storage và phân phối qua CDN.

Ví dụ:

```
cdn.playnest.zone/  
  assets/  
  games/  
  levels/  
  audio/
```

Nên tạo asset manifest:

```
{  
  "cat_hungry": {  
    "url": "/assets/tricky-brain/cat-hungry.webp",  
    "type": "image",  
    "width": 512,  
    "height": 512,  
    "version": "1.0.2"  
  }  
}
```

Ưu tiên định dạng:

```
WebP cho ảnh  
AVIF cho background lớn  
WebM hoặc spritesheet cho animation  
MP3 hoặc OGG cho audio
```

12. Offline-first và PWA

Game nên có thể chơi sau lần tải đầu ngay cả khi mất mạng.

Thành phần

```
Service Worker
Cache Storage
IndexedDB
Asset Manifest
Level Cache
Background Sync
```

Luồng hoạt động

Lần mở đầu tiên

Browser

```
|
|— Tải game shell
|— Tải engine
|— Tải 20 level đầu
|— Tải asset cần thiết
|— Lưu cache
```

Các lần mở sau

Browser

```
|
|— Đọc game shell từ cache
|— Đọc level từ IndexedDB
|— Chơi ngay
```

Khi có mạng:

- Đồng bộ tiến trình.
- Tải thêm level.
- Gửi analytics.
- Kiểm tra remote config.
- Hiển thị quảng cáo.

Khi mất mạng:

- Game vẫn chạy.
- Lưu tiến trình local.
- Không hiển thị quảng cáo online.
- Đợi có mạng để đồng bộ.

13. Save Game

Local save

Dùng IndexedDB để lưu:

```
Level hiện tại
Level đã hoàn thành
Số lần thử
Hint đã sử dụng
Cài đặt âm thanh
Ngôn ngữ
Achievement tạm thời
Analytics chưa gửi
```

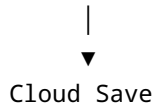
Ví dụ:

```
{
  "gameId": "tricky-brain",
  "currentLevel": 125,
  "completedLevels": [1, 2, 3, 4, 5],
  "hintBalance": 4,
  "soundEnabled": true,
  "lastPlayedAt": "2026-07-28T03:00:00Z"
}
```

Cloud save

Khi người dùng đăng nhập:

```
Local Save
  |
  ▼
Conflict Resolver
```



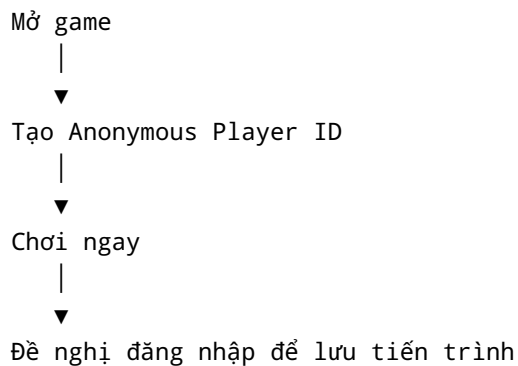
Nguyên tắc mặc định:

- Tiến trình cao hơn được giữ lại.
- Vật phẩm mua bằng tiền thật ưu tiên dữ liệu server.
- Dữ liệu achievement được hợp nhất.

14. Authentication

Không nên bắt người dùng đăng nhập trước khi chơi.

Luồng đề xuất:

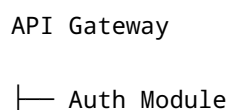


Các phương thức:

- Google.
- Apple.
- Email magic link.
- Anonymous account.

Sau khi đăng nhập, anonymous account được liên kết với tài khoản chính.

15. Backend modules



- └─ User Module
- └─ Game Module
- └─ Level Module
- └─ Progress Module
- └─ Inventory Module
- └─ Achievement Module
- └─ Leaderboard Module
- └─ Remote Config Module
- └─ Analytics Module
- └─ Payment Module
- └─ Admin Module

16. Cơ sở dữ liệu

Bảng users

```
id
anonymous_id
email
display_name
avatar_url
created_at
last_active_at
```

Bảng games

```
id
slug
name
description
status
current_version
created_at
```

Bảng levels

```
id
game_id
level_number
version
```

```
data_url
difficulty
status
published_at
```

Bảng player_progress

```
user_id
game_id
current_level
completed_levels
hint_balance
updated_at
```

Bảng level_attempts

```
id
user_id
game_id
level_id
attempt_number
duration_ms
success
hint_used
created_at
```

Bảng achievements

```
id
game_id
code
name
condition
reward
```

Bảng user_achievements

```
user_id
achievement_id
unlocked_at
```

17. Analytics

Analytics đặc biệt quan trọng với game casual.

Mỗi level cần ghi lại:

```
level_started
level_first_interaction
level_failed
level_hint_opened
level_hint_used
level_completed
level_abandoned
ad_requested
ad_started
ad_completed
ad_failed
```

Ví dụ event:

```
{
  "event": "level_completed",
  "gameId": "tricky-brain",
  "levelId": "tricky-brain-125",
  "attempts": 3,
  "durationMs": 18200,
  "hintUsed": false,
  "platform": "mobile-web"
}
```

Chỉ số cần theo dõi

- Tỷ lệ hoàn thành từng level.
- Thời gian trung bình.
- Số lần thử.
- Tỷ lệ dùng hint.
- Tỷ lệ thoát.
- Mức độ khó.
- Retention D1, D7, D30.
- Session length.
- Số level mỗi phiên.
- Doanh thu mỗi người dùng.

- Doanh thu mỗi nghìn phiên.
-

18. Difficulty Scoring

Có thể tự động đánh giá độ khó:

```
Difficulty Score =  
    Completion Time  
+ Number of Attempts  
+ Hint Usage  
+ Abandon Rate
```

Ví dụ:

```
Easy:  
- Hoàn thành dưới 15 giây  
- Tỷ lệ hoàn thành trên 85%  
  
Medium:  
- Hoàn thành 15-35 giây  
- Tỷ lệ hoàn thành 60-85%  
  
Hard:  
- Hoàn thành trên 35 giây  
- Tỷ lệ hoàn thành dưới 60%
```

Nếu một level quá khó, hệ thống có thể:

- Hiển thị hint sớm hơn.
 - Đưa level sang vị trí khác.
 - Thay đổi câu hỏi.
 - Chạy A/B test.
 - Tạm ẩn level.
-

19. Monetization

19.1 Quảng cáo

Web có thể dùng:

- Google AdSense cho giao diện website.
- Google Ad Manager khi quy mô lớn.
- Rewarded ads từ nền tảng hỗ trợ game web.
- Quảng cáo nội bộ giữa các game.

Vị trí quảng cáo:

Trang danh sách game
Trang kết quả level
Sau một số level nhất định
Rewarded ad để lấy hint
Rewarded ad để bỏ qua level
Banner ngoài vùng canvas
Native ad trong trang game

Không nên hiển thị quảng cáo che gameplay liên tục.

19.2 Quy tắc interstitial

Ví dụ:

Không quảng cáo trong 3 level đầu
Sau đó mỗi 4-6 level
Không quảng cáo nếu phiên chơi dưới 60 giây
Không quảng cáo hai lần trong vòng 90 giây
Không quảng cáo ngay sau rewarded ad

Các thông số này phải điều khiển bằng Remote Config.

19.3 Subscription hoặc IAP

Có thể bán:

Remove Ads
Unlimited Hints
Premium Game Pack

Daily Bonus
Exclusive Levels

Thanh toán web:

Stripe
Paddle
Google Play Billing nếu đóng gói Android
Apple IAP nếu đóng gói iOS

20. Remote Config

Không nên hard-code monetization hoặc gameplay rule.

Remote Config quản lý:

```
{
  "ads": {
    "enabled": true,
    "firstInterstitialAfterLevel": 5,
    "interstitialInterval": 4,
    "minimumSecondsBetweenAds": 90
  },
  "hints": {
    "freeHintsPerDay": 3,
    "rewardedHintEnabled": true
  },
  "gameplay": {
    "skipLevelEnabled": true,
    "dailyChallengeEnabled": true
  }
}
```

Điều này cho phép thay đổi hệ thống mà không cần deploy lại frontend.

21. CDN và phân phối nội dung

Không nên tải toàn bộ hàng nghìn level ngay lần đầu.

Chia level thành pack:

Pack 1: Level 1-20
Pack 2: Level 21-50
Pack 3: Level 51-100
Pack 4: Level 101-150

Luồng tải:

Người chơi đang ở level 15
↓
Tự động preload pack 21-50

CDN phân phối:

- File JavaScript.
- Hình ảnh.
- Audio.
- Level JSON.
- Game manifest.
- Animation.

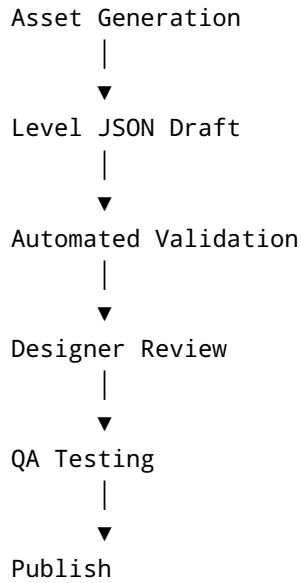
Có thể dùng:

Cloudflare CDN
Cloudflare R2
AWS CloudFront + S3
Bunny CDN

22. AI Content Pipeline

AI có thể hỗ trợ tạo nội dung nhưng không nên publish tự động.

Prompt Template
↓
AI Level Idea Generator
↓
Structured Level Spec
↓



AI hỗ trợ:

- Sinh ý tưởng màn chơi.
- Viết câu hỏi.
- Viết hint.
- Sinh biến thể.
- Dịch đa ngôn ngữ.
- Sinh asset mẫu.
- Phân tích độ khó.
- Phát hiện level trùng ý tưởng.

Con người vẫn cần kiểm tra:

- Logic có hợp lý không.
- Đáp án có gây ức chế không.
- Asset có nhất quán không.
- Nội dung có vi phạm bản quyền không.
- Level có lỗi thao tác không.

23. Đa ngôn ngữ

Tách text khỏi dữ liệu gameplay.

Ví dụ level chỉ lưu key:

```
{  
  "questionKey": "level.tricky_brain_001.question",  
}
```

```
"hintKey": "level.tricky_brain_001.hint"
}
```

File tiếng Việt:

```
{
  "level.tricky_brain_001.question": "Hãy giúp chú mèo lấy con cá",
  "level.tricky_brain_001.hint": "Thử kéo con cá lại gần chú mèo"
}
```

File tiếng Anh:

```
{
  "level.tricky_brain_001.question": "Help the cat get the fish",
  "level.tricky_brain_001.hint": "Try moving the fish closer to the cat"
}
```

Ưu tiên hỗ trợ:

English
Vietnamese
Spanish
Portuguese
Indonesian
Hindi
Turkish
German
French
Japanese
Korean

24. Hệ thống nhiều game

Mỗi game là một package riêng nhưng dùng chung PlayNest SDK.

PlayNest SDK

- └─ Auth
- └─ Cloud Save
- └─ Analytics

- └─ Ads
- └─ Payments
- └─ Achievements
- └─ Leaderboard
- └─ Remote Config
- └─ Audio Settings
- └─ Localization

Mỗi game chỉ cần tập trung vào gameplay riêng.

Ví dụ:

```
playnest.zone/games/tricky-brain
playnest.zone/games/word-connect
playnest.zone/games/emoji-quiz
playnest.zone/games/find-hidden
```

Có thể hỗ trợ subdomain:

```
brain.playnest.zone
words.playnest.zone
emoji.playnest.zone
```

Tuy nhiên, giai đoạn đầu nên dùng đường dẫn chung để quản lý SEO và analytics dễ hơn.

25. Bảo mật và chống gian lận

Với game casual offline, không thể tin hoàn toàn dữ liệu client.

Dữ liệu quan trọng phải kiểm tra server:

- Điểm leaderboard.
- Vật phẩm mua.
- Reward từ quảng cáo.
- Achievement đặc biệt.
- Daily reward.
- Thanh toán.

Biện pháp:

Signed save data
Request signature
Rate limiting
Server timestamp
Replay protection
Ad reward verification
Payment webhook verification

Không cần chống cheat quá phức tạp cho tiến trình level thông thường.

26. Kiểm thử

Unit Test

Kiểm tra:

- Rule engine.
- Condition evaluator.
- Save manager.
- Level parser.
- Analytics events.

Integration Test

Kiểm tra:

- Load level.
- Hoàn thành level.
- Dùng hint.
- Save và restore.
- Đồng bộ cloud.

Visual Test

Chụp screenshot trên:

iPhone
Android
Tablet
Desktop

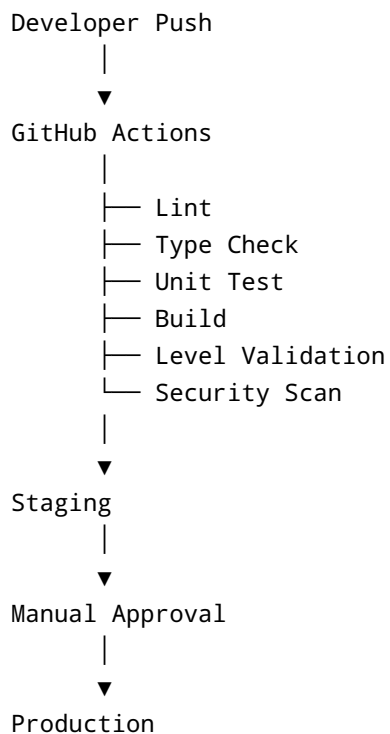
Màn hình nhỏ
Màn hình siêu dài

Automated Level Test

Mỗi level cần kiểm tra:

- Asset có tồn tại.
- Object ID có trùng không.
- Rule có tham chiếu đúng object không.
- Level có action complete không.
- Hint có tồn tại không.
- Text có vượt màn hình không.

27. CI/CD



Triển khai:

Frontend: Vercel hoặc Cloudflare Pages
API: Docker trên VPS hoặc managed container
Database: Managed PostgreSQL

Redis: Managed Redis
Assets: Object Storage + CDN

28. Monitoring

Cần theo dõi:

Frontend errors
Game crashes
Asset load failures
API latency
Database latency
Ad errors
Level completion anomalies
CDN cache hit
Web Vitals

Công cụ:

Sentry
OpenTelemetry
Grafana
Prometheus
PostHog
Google Analytics

29. Lộ trình phát triển

Phase 1 – Game prototype

Thời gian mục tiêu: 2–4 tuần.

Phạm vi:

- Phaser runtime.
- 10 level viết bằng code.
- Tap, drag, drop.
- Responsive mobile.
- Lưu tiến trình local.

- Không backend.
- Không tài khoản.
- Không editor.

Mục tiêu là kiểm chứng gameplay.

Phase 2 – Puzzle Engine

Phạm vi:

- Level JSON.
- Object factory.
- Interaction engine.
- Rule engine.
- Hint system.
- 50 level.
- Analytics cơ bản.
- PWA offline.

Mục tiêu là không cần viết code riêng cho phần lớn level.

Phase 3 – Level Editor

Phạm vi:

- Canvas editor.
- Asset library.
- Rule builder.
- Preview.
- Publish level.
- Versioning.
- Validation.

Mục tiêu là designer tự tạo màn chơi.

Phase 4 – Backend Platform

Phạm vi:

- Anonymous user.
- Login.
- Cloud save.
- Remote config.

- Level distribution.
 - Admin dashboard.
 - Analytics dashboard.
 - Daily challenge.
-

Phase 5 – Monetization

Phạm vi:

- Banner.
 - Interstitial.
 - Rewarded hint.
 - Remove ads.
 - A/B testing.
 - Frequency cap.
 - Revenue analytics.
-

Phase 6 – Multi-game Platform

Phạm vi:

- Game library.
 - Shared user profile.
 - Achievement.
 - Cross-game recommendation.
 - Shared wallet.
 - Game SDK.
 - Thêm Word Game và Hidden Object.
-

30. MVP đề xuất

MVP không nên làm toàn bộ hệ thống ngay.

Phạm vi hợp lý:

```
1 game
50 level
1 puzzle engine
1 web application
1 local save
1 hint system
```

```
1 analytics layer
1 màn hình chọn level
1 PWA
```

Chưa cần:

- Login.
- Cloud save.
- Leaderboard.
- Payment.
- AI generator hoàn chỉnh.
- Microservices.
- Multiplayer.
- Editor quá phức tạp.

Stack MVP:

```
React hoặc Next.js
Phaser 3
TypeScript
Zustand
IndexedDB
PWA
Cloudflare Pages
Cloudflare R2
PostHog hoặc Firebase Analytics
Sentry
```

31. Kiến trúc MVP cụ thể

```
playnest-mvp/
├─ src/
│  ├─ app/
│  │  ├─ home
│  │  ├─ games
│  │  └─ settings
│  └─ game/
│     ├─ config
│     ├─ scenes
│     ├─ engine
│     └─ LevelLoader.ts
```

```
| | | | | ObjectFactory.ts
| | | | | InteractionEngine.ts
| | | | | RuleEngine.ts
| | | | | HintEngine.ts
| | | | | SaveManager.ts
| | | | |
| | | | | components
| | | | |
| | | | | levels/
| | | | | | level-001.json
| | | | | | level-002.json
| | | | | | manifest.json
| | | | |
| | | | | assets/
| | | | | analytics/
| | | | | storage/
| | | | | shared/
| | | | |
| | | | | public/
| | | | | | assets
| | | | | | audio
| | | | | | manifest.webmanifest
```

32. Quyết định kiến trúc quan trọng

Dùng Phaser thay vì Unity WebGL

Lý do:

- Bundle nhỏ hơn.
- Load nhanh hơn.
- Tối ưu mobile browser tốt hơn.
- Dễ tích hợp React.
- Dễ làm level editor.
- Dễ deploy.
- Không cần runtime Unity lớn.

Dùng JSON level thay vì viết từng level bằng code

Lý do:

- Designer có thể tạo nội dung.
- Dễ dịch.

- Dễ cập nhật.
- Dễ A/B test.
- Dễ để AI sinh level.
- Có thể tải level mà không cập nhật game.

Offline-first

Lý do:

- Người dùng vào game nhanh.
- Mạng yếu vẫn chơi được.
- Giảm phụ thuộc backend.
- Giảm chi phí server.
- Tăng retention.

Modular monolith trước, microservices sau

Giai đoạn đầu chỉ cần một backend dạng modular monolith.

Không nên tách microservice khi chưa có lượng người dùng đủ lớn.

33. Kết luận

Kiến trúc phù hợp nhất cho PlayNest là:

```
Next.js
+
Phaser 3
+
Data-driven Puzzle Engine
+
JSON Level
+
Web Level Editor
+
PWA Offline
+
NestJS Backend
+
PostgreSQL
+
Object Storage và CDN
```

Lợi thế cốt lõi của hệ thống không nằm ở đồ họa hay backend phức tạp, mà nằm ở ba thành phần:

Puzzle Engine
Level Editor
Content Pipeline

Khi ba thành phần này hoạt động tốt, PlayNest có thể:

- Tạo hàng nghìn level.
- Phát triển nhiều game từ cùng một engine.
- Cho designer tạo màn chơi không cần code.
- Dùng AI để tăng tốc sản xuất nội dung.
- Chạy trên web, PWA, Android và iOS.
- Kiếm tiền từ quảng cáo và gói premium.
- Mở rộng thành một hệ sinh thái casual game hoàn chỉnh.